

RL_03_sb3

April 10, 2026

1 Reinforcement Learning - Parte 3: Stable-Baselines3

Prerequisites: Haber completado las Partes 1 y 2

1.1 Serie Completa

Notebook	Contenido	Estado
RL_01_fundamentos.ipynb	Fundamentos, SARSA, Q-Learning	Completado
RL_02_dqn.ipynb	DQN, Gymnasium, Práctica	Completado
RL_03_sb3.ipynb (este)	Stable-Baselines3, PPO, SAC	Estás aquí

1.2 ¿Qué aprendimos antes?

- **Parte 1:** Conceptos fundamentales, SARSA, Q-Learning tabular
- **Parte 2:** DQN (redes neuronales), Gymnasium, implementación desde cero

1.3 ¿Qué aprenderemos aquí?

- **Stable-Baselines3:** La librería estándar para entrenar agentes RL
- **Algoritmos avanzados:** PPO, A2C, SAC, TD3
- **Entrenamiento práctico:** Callbacks, logging, guardado de modelos
- **Proyectos reales:** Conexión con 04_proyectos_avanzados/

1.4 Índice

1. ¿Por qué Stable-Baselines3?
2. Instalación y Setup
3. Tu Primer Agente con SB3
4. Algoritmos Disponibles
5. Políticas (Redes Neuronales)
6. Entrenamiento Avanzado
7. Callbacks y Logging
8. Guardar y Cargar Modelos
9. Evaluación de Agentes
10. Hiperparámetros

11. Proyectos Prácticos

```
[1]: # INSTALACIÓN
# =====
# Ejecutar solo si no están instaladas las dependencias

# Stable-Baselines3 (incluye PyTorch si no lo tienes)
!pip install stable-baselines3

# Para entornos Box2D (LunarLander, BipedalWalker)
!pip install "gymnasium[box2d]"

# Para visualización de entrenamiento
!pip install tensorboard

# Instalar pygame
!pip install pygame --break-system-packages

# 3. Instalar gymnasium con soporte classic-control
!pip install "gymnasium[classic-control]" --break-system-packages
```

```
/bin/bash: /home/jeironpro/Documentos/OPT-ML2/.venv/bin/pip:
/home/jeironpro/OPT-ML/.venv/bin/python: intérprete erróneo: No existe el
fichero o el directorio

/bin/bash: /home/jeironpro/Documentos/OPT-ML2/.venv/bin/pip:
/home/jeironpro/OPT-ML/.venv/bin/python: intérprete erróneo: No existe el
fichero o el directorio

/bin/bash: /home/jeironpro/Documentos/OPT-ML2/.venv/bin/pip:
/home/jeironpro/OPT-ML/.venv/bin/python: intérprete erróneo: No existe el
fichero o el directorio

/bin/bash: /home/jeironpro/Documentos/OPT-ML2/.venv/bin/pip:
/home/jeironpro/OPT-ML/.venv/bin/python: intérprete erróneo: No existe el
fichero o el directorio

/bin/bash: /home/jeironpro/Documentos/OPT-ML2/.venv/bin/pip:
/home/jeironpro/OPT-ML/.venv/bin/python: intérprete erróneo: No existe el
fichero o el directorio
```

```
[2]: # IMPORTS
# =====

# Stable-Baselines3
try:
    from stable_baselines3 import PPO, A2C, DQN, SAC, TD3
    from stable_baselines3.common.env_util import make_vec_env
    from stable_baselines3.common.evaluation import evaluate_policy
    from stable_baselines3.common.callbacks import EvalCallback,
    ↳ CheckpointCallback
```

```

from stable_baselines3.common.monitor import Monitor
SB3_AVAILABLE = True
print(" Stable-Baselines3 disponible")
except ImportError:
    SB3_AVAILABLE = False
    print(" Stable-Baselines3 no instalado. Ejecuta: pip install_
↳stable-baselines3[extra]")

# Gymnasium
import gymnasium as gym
print(" Gymnasium disponible")

# Utilidades
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
import os

# Configuración para Windows
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'

print("\n Imports completados")

```

Stable-Baselines3 disponible
Gymnasium disponible

Imports completados

```

[3]: # TU PRIMER AGENTE EN 3 LÍNEAS
# =====

if SB3_AVAILABLE:
    # 1. CREAR EL MODELO
    # - PPO: Algoritmo (Proximal Policy Optimization)
    # - "MlpPolicy": Red neuronal fully-connected (Multi-Layer Perceptron)
    # - "CartPole-v1": Entorno de Gymnasium
    model = PPO("MlpPolicy", "CartPole-v1", verbose=1)

    # 2. ENTRENAR
    # - total_timesteps: Número total de pasos de interacción
    # - CartPole se resuelve en ~20,000 pasos con PPO
    model.learn(total_timesteps=20_000)

    # 3. EVALUAR
    # - Crea un entorno nuevo para evaluar
    # - Ejecuta 10 episodios y calcula estadísticas
    env = gym.make("CartPole-v1")

```

```

mean_reward, std_reward = evaluate_policy(model, env, n_eval_episodes=10)
print(f"\n Recompensa media: {mean_reward:.2f} ± {std_reward:.2f}")
print(f" (Máximo posible en CartPole: 500)")
env.close()
else:
    print(" Instala stable-baselines3 para ejecutar este código")

```

Using cpu device

Creating environment from the given name 'CartPole-v1'

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

```

-----
| rollout/          |          |
|   ep_len_mean    | 21.4     |
|   ep_rew_mean    | 21.4     |
| time/            |          |
|   fps            | 1703     |
|   iterations      | 1         |
|   time_elapsed    | 1         |
|   total_timesteps | 2048     |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 26.8     |
|   ep_rew_mean    | 26.8     |
| time/            |          |
|   fps            | 925      |
|   iterations      | 2         |
|   time_elapsed    | 4         |
|   total_timesteps | 4096     |
| train/           |          |
|   approx_kl       | 0.009766517 |
|   clip_fraction   | 0.134     |
|   clip_range      | 0.2       |
|   entropy_loss     | -0.685    |
|   explained_variance | -0.00845  |
|   learning_rate    | 0.0003    |
|   loss             | 7.14      |
|   n_updates        | 10        |
|   policy_gradient_loss | -0.0208   |
|   value_loss       | 50.1      |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 33.2     |
|   ep_rew_mean    | 33.2     |
| time/            |          |
|   fps            | 764      |

```

	iterations	3	
	time_elapsed	8	
	total_timesteps	6144	
	train/		
	approx_kl	0.010200074	
	clip_fraction	0.0603	
	clip_range	0.2	
	entropy_loss	-0.667	
	explained_variance	0.112	
	learning_rate	0.0003	
	loss	12.2	
	n_updates	20	
	policy_gradient_loss	-0.0172	
	value_loss	35	

	rollout/		
	ep_len_mean	45.6	
	ep_rew_mean	45.6	
	time/		
	fps	711	
	iterations	4	
	time_elapsed	11	
	total_timesteps	8192	
	train/		
	approx_kl	0.009635636	
	clip_fraction	0.1	
	clip_range	0.2	
	entropy_loss	-0.634	
	explained_variance	0.289	
	learning_rate	0.0003	
	loss	20.7	
	n_updates	30	
	policy_gradient_loss	-0.0225	
	value_loss	46.1	

	rollout/		
	ep_len_mean	59.3	
	ep_rew_mean	59.3	
	time/		
	fps	681	
	iterations	5	
	time_elapsed	15	
	total_timesteps	10240	
	train/		
	approx_kl	0.008387031	
	clip_fraction	0.051	

clip_range	0.2	
entropy_loss	-0.621	
explained_variance	0.253	
learning_rate	0.0003	
loss	24.6	
n_updates	40	
policy_gradient_loss	-0.014	
value_loss	65.1	

rollout/		
ep_len_mean	75.1	
ep_rew_mean	75.1	
time/		
fps	652	
iterations	6	
time_elapsed	18	
total_timesteps	12288	
train/		
approx_kl	0.0074563995	
clip_fraction	0.0563	
clip_range	0.2	
entropy_loss	-0.601	
explained_variance	0.459	
learning_rate	0.0003	
loss	28.1	
n_updates	50	
policy_gradient_loss	-0.0159	
value_loss	67.4	

rollout/		
ep_len_mean	90.5	
ep_rew_mean	90.5	
time/		
fps	641	
iterations	7	
time_elapsed	22	
total_timesteps	14336	
train/		
approx_kl	0.006591262	
clip_fraction	0.071	
clip_range	0.2	
entropy_loss	-0.585	
explained_variance	0.604	
learning_rate	0.0003	
loss	30.4	
n_updates	60	

	policy_gradient_loss		-0.0126	
	value_loss		55	

	rollout/			
	ep_len_mean		108	
	ep_rew_mean		108	
	time/			
	fps		635	
	iterations		8	
	time_elapsed		25	
	total_timesteps		16384	
	train/			
	approx_kl		0.005216561	
	clip_fraction		0.0473	
	clip_range		0.2	
	entropy_loss		-0.577	
	explained_variance		0.725	
	learning_rate		0.0003	
	loss		14.9	
	n_updates		70	
	policy_gradient_loss		-0.0067	
	value_loss		43.2	

	rollout/			
	ep_len_mean		126	
	ep_rew_mean		126	
	time/			
	fps		631	
	iterations		9	
	time_elapsed		29	
	total_timesteps		18432	
	train/			
	approx_kl		0.003455808	
	clip_fraction		0.0129	
	clip_range		0.2	
	entropy_loss		-0.568	
	explained_variance		0.871	
	learning_rate		0.0003	
	loss		4.58	
	n_updates		80	
	policy_gradient_loss		-0.00443	
	value_loss		25.1	

	rollout/			
	ep_len_mean		148	

	ep_rew_mean	148	
	time/		
	fps	627	
	iterations	10	
	time_elapsed	32	
	total_timesteps	20480	
	train/		
	approx_kl	0.0048525874	
	clip_fraction	0.0338	
	clip_range	0.2	
	entropy_loss	-0.578	
	explained_variance	0.798	
	learning_rate	0.0003	
	loss	1.67	
	n_updates	90	
	policy_gradient_loss	-0.00531	
	value_loss	18.4	

```
/home/jeironpro/Documentos/OPT-ML2/.venv/lib64/python3.11/site-
packages/stable_baselines3/common/evaluation.py:70: UserWarning: Evaluation
environment is not wrapped with a ``Monitor`` wrapper. This may result in
reporting modified episode lengths and rewards, if other wrappers happen to
modify these. Consider wrapping environment first with ``Monitor`` wrapper.
warnings.warn(
```

```
Recompensa media: 500.00 ± 0.00
(Máximo posible en CartPole: 500)
```

```
[4]: # VER EL AGENTE EN ACCIÓN
# =====

if SB3_AVAILABLE:
    # Crear entorno con renderizado
    env = gym.make("CartPole-v1", render_mode="human")

    # Ejecutar un episodio
    obs, info = env.reset()
    total_reward = 0

    for step in range(500):
        # El modelo predice la acción
        action, _ = model.predict(obs, deterministic=True)

        # Ejecutar acción
        obs, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
```



```

        if terminated or truncated:
            break

    print(f"\n Episodio completado: {total_reward} pasos")
    env.close()

```

Episodio completado: 500.0 pasos

```

[5]: # COMPARACIÓN DE ALGORITMOS
# =====
# Entrenamos varios algoritmos en el mismo entorno y comparamos

if SB3_AVAILABLE:
    from stable_baselines3 import PPO, A2C, DQN

    # Entorno simple para comparar
    env_id = "CartPole-v1"
    timesteps = 10_000 # Pocos pasos para demo rápida

    resultados = {}

    # PPO
    print("Entrenando PPO...")
    model_ppo = PPO("MlpPolicy", env_id, verbose=0)
    model_ppo.learn(total_timesteps=timesteps)
    mean_r, _ = evaluate_policy(model_ppo, gym.make(env_id),
    ↪n_eval_episodes=10, warn=False)
    resultados["PPO"] = mean_r
    print(f" PPO: {mean_r:.1f}")

    # A2C
    print("Entrenando A2C...")
    model_a2c = A2C("MlpPolicy", env_id, verbose=0)
    model_a2c.learn(total_timesteps=timesteps)
    mean_r, _ = evaluate_policy(model_a2c, gym.make(env_id),
    ↪n_eval_episodes=10, warn=False)
    resultados["A2C"] = mean_r
    print(f" A2C: {mean_r:.1f}")

    # DQN
    print("Entrenando DQN...")
    model_dqn = DQN("MlpPolicy", env_id, verbose=0)
    model_dqn.learn(total_timesteps=timesteps)
    mean_r, _ = evaluate_policy(model_dqn, gym.make(env_id),
    ↪n_eval_episodes=10, warn=False)

```

```

resultados["DQN"] = mean_r
print(f" DQN: {mean_r:.1f}")

# Visualizar
plt.figure(figsize=(8, 4))
plt.bar(resultados.keys(), resultados.values(), color=['blue', 'green', 'orange'])
plt.ylabel("Recompensa Media")
plt.title(f"Comparación de Algoritmos en {env_id} ({timesteps:,} pasos)")
plt.axhline(y=500, color='red', linestyle='--', label='Máximo')
plt.legend()
plt.show()

```

Entrenando PPO...

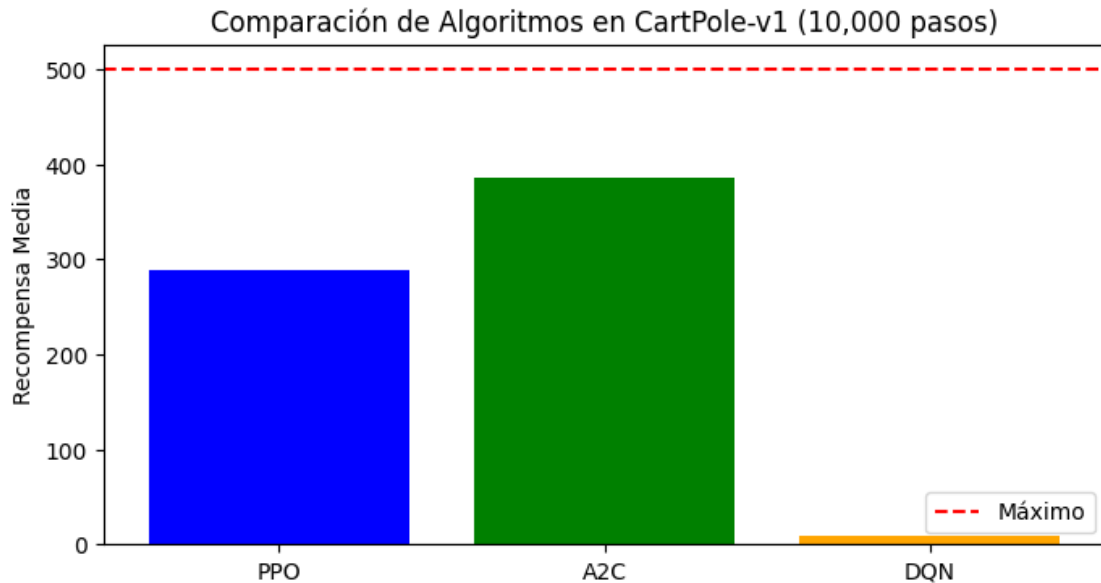
PPO: 289.3

Entrenando A2C...

A2C: 385.3

Entrenando DQN...

DQN: 8.8



```

[6]: # PERSONALIZAR LA ARQUITECTURA
# =====

if SB3_AVAILABLE:
    # Definir arquitectura custom
    # - net_arch: Lista de capas para policy y value networks
    # - [dict(pi=[...], vf=[...])] separa las arquitecturas

```

```

policy_kwargs = dict(
    net_arch=[128, 128, 64]  # 3 capas: 128, 128, 64 neuronas
)

# Crear modelo con arquitectura custom
model = PPO(
    "MlpPolicy",
    "CartPole-v1",
    policy_kwargs=policy_kwargs,
    verbose=1
)

print("Arquitectura de la política:")
print(model.policy)

```

Using cpu device

Creating environment from the given name 'CartPole-v1'

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

Arquitectura de la política:

```

ActorCriticPolicy(
  (features_extractor): FlattenExtractor(
    (flatten): Flatten(start_dim=1, end_dim=-1)
  )
  (pi_features_extractor): FlattenExtractor(
    (flatten): Flatten(start_dim=1, end_dim=-1)
  )
  (vf_features_extractor): FlattenExtractor(
    (flatten): Flatten(start_dim=1, end_dim=-1)
  )
  (mlp_extractor): MlpExtractor(
    (policy_net): Sequential(
      (0): Linear(in_features=4, out_features=128, bias=True)
      (1): Tanh()
      (2): Linear(in_features=128, out_features=128, bias=True)
      (3): Tanh()
      (4): Linear(in_features=128, out_features=64, bias=True)
      (5): Tanh()
    )
  )
  (value_net): Sequential(
    (0): Linear(in_features=4, out_features=128, bias=True)
    (1): Tanh()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): Tanh()
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): Tanh()
  )
)

```

```
)
(action_net): Linear(in_features=64, out_features=2, bias=True)
(value_net): Linear(in_features=64, out_features=1, bias=True)
)
```

```
[7]: # ARQUITECTURAS SEPARADAS PARA ACTOR Y CRITIC
# =====

if SB3_AVAILABLE:
    # En algoritmos Actor-Critic (PPO, A2C), puedes separar las arquitecturas

    policy_kwargs = dict(
        net_arch=dict(
            pi=[64, 64],    # Actor (política): 2 capas de 64
            vf=[128, 128]   # Critic (valor): 2 capas de 128
        )
    )

    model = PPO(
        "MlpPolicy",
        "CartPole-v1",
        policy_kwargs=policy_kwargs,
        verbose=0
    )

    print(" Modelo con arquitecturas separadas creado")
    print(f" Actor: 2 capas de 64")
    print(f" Critic: 2 capas de 128")
```

```
Modelo con arquitecturas separadas creado
Actor: 2 capas de 64
Critic: 2 capas de 128
```

```
[8]: # ENTRENAMIENTO CON ENTORNOS PARALELOS
# =====

if SB3_AVAILABLE:
    from stable_baselines3.common.env_util import make_vec_env

    # Crear 4 entornos paralelos
    # - Cada entorno recolecta experiencias simultáneamente
    # - El entrenamiento es ~4x más rápido
    vec_env = make_vec_env("CartPole-v1", n_envs=4)

    # PPO funciona especialmente bien con múltiples entornos
    model = PPO(
        "MlpPolicy",
        vec_env,
```

```

        verbose=1,
        n_steps=1024,      # Pasos por entorno antes de actualizar
        batch_size=64,     # Tamaño del minibatch
        n_epochs=10,       # Épocas de optimización por actualización
    )

    print(f"\n Entrenando con {vec_env.num_envs} entornos paralelos...")
    model.learn(total_timesteps=20_000)

    # Evaluar
    mean_r, std_r = evaluate_policy(model, gym.make("CartPole-v1"),
    ↪n_eval_episodes=10)
    print(f"\n Resultado: {mean_r:.1f} ± {std_r:.1f}")

```

Using cpu device

Entrenando con 4 entornos paralelos...

```

-----
| rollout/          |          |
|   ep_len_mean    | 21.7     |
|   ep_rew_mean    | 21.7     |
| time/            |          |
|   fps            | 2619     |
|   iterations     | 1        |
|   time_elapsed   | 1        |
|   total_timesteps | 4096     |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 30.9     |
|   ep_rew_mean    | 30.9     |
| time/            |          |
|   fps            | 1034     |
|   iterations     | 2        |
|   time_elapsed   | 7        |
|   total_timesteps | 8192     |
| train/           |          |
|   approx_kl      | 0.013384671 |
|   clip_fraction  | 0.16     |
|   clip_range     | 0.2      |
|   entropy_loss   | -0.684   |
|   explained_variance | -0.000168 |
|   learning_rate  | 0.0003   |
|   loss           | 3.65     |
|   n_updates      | 10       |
|   policy_gradient_loss | -0.0223  |
|   value_loss     | 31.5     |
-----

```

rollout/		
ep_len_mean	49.8	
ep_rew_mean	49.8	
time/		
fps	882	
iterations	3	
time_elapsed	13	
total_timesteps	12288	
train/		
approx_kl	0.013605465	
clip_fraction	0.155	
clip_range	0.2	
entropy_loss	-0.653	
explained_variance	0.313	
learning_rate	0.0003	
loss	9.78	
n_updates	20	
policy_gradient_loss	-0.0266	
value_loss	29.7	

rollout/		
ep_len_mean	65.1	
ep_rew_mean	65.1	
time/		
fps	817	
iterations	4	
time_elapsed	20	
total_timesteps	16384	
train/		
approx_kl	0.008852183	
clip_fraction	0.106	
clip_range	0.2	
entropy_loss	-0.616	
explained_variance	0.396	
learning_rate	0.0003	
loss	23.3	
n_updates	30	
policy_gradient_loss	-0.0226	
value_loss	42	

rollout/		
ep_len_mean	90.3	
ep_rew_mean	90.3	
time/		
fps	788	

iterations	5
time_elapsed	25
total_timesteps	20480
train/	
approx_kl	0.011278821
clip_fraction	0.14
clip_range	0.2
entropy_loss	-0.587
explained_variance	0.559
learning_rate	0.0003
loss	26.9
n_updates	40
policy_gradient_loss	-0.0244
value_loss	47.4

```
/home/jeironpro/Documentos/OPT-ML2/.venv/lib64/python3.11/site-
packages/stable_baselines3/common/evaluation.py:70: UserWarning: Evaluation
environment is not wrapped with a ``Monitor`` wrapper. This may result in
reporting modified episode lengths and rewards, if other wrappers happen to
modify these. Consider wrapping environment first with ``Monitor`` wrapper.
warnings.warn(
```

Resultado: 420.4 ± 75.3

```
[9]: # CALLBACKS BÁSICOS
# =====

if SB3_AVAILABLE:
    from stable_baselines3.common.callbacks import (
        EvalCallback,
        CheckpointCallback,
        CallbackList
    )

    # Directorio para guardar
    log_dir = "./logs/"
    os.makedirs(log_dir, exist_ok=True)

    # 1. CheckpointCallback: Guarda el modelo cada N pasos
    checkpoint_callback = CheckpointCallback(
        save_freq=5000,           # Guardar cada 5000 pasos
        save_path=log_dir,       # Directorio
        name_prefix="ppo_cartpole" # Prefijo del archivo
    )

    # 2. EvalCallback: Evalúa y guarda el mejor modelo
```

```

eval_env = gym.make("CartPole-v1")
eval_callback = EvalCallback(
    eval_env,
    best_model_save_path=log_dir,    # Guarda el mejor
    log_path=log_dir,               # Logs de evaluación
    eval_freq=2000,                 # Evaluar cada 2000 pasos
    n_eval_episodes=5,              # 5 episodios por evaluación
    deterministic=True
)

# Combinar callbacks
callbacks = CallbackList([checkpoint_callback, eval_callback])

# Entrenar con callbacks
model = PPO("MlpPolicy", "CartPole-v1", verbose=1)
model.learn(total_timesteps=20_000, callback=callbacks)

print(f"\n Modelos guardados en {log_dir}")
print(f" Archivos: {os.listdir(log_dir)}")

```

Using cpu device

Creating environment from the given name 'CartPole-v1'

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

Eval num_timesteps=2000, episode_reward=46.20 +/- 13.38

Episode length: 46.20 +/- 13.38

```

-----
| eval/                |                |
|   mean_ep_length    | 46.2           |
|   mean_reward       | 46.2           |
| time/                |                |
|   total_timesteps   | 2000           |
-----

```

New best mean reward!

```

-----
| rollout/             |                |
|   ep_len_mean       | 22.2           |
|   ep_rew_mean       | 22.2           |
| time/               |                |
|   fps               | 1355           |
|   iterations        | 1              |
|   time_elapsed      | 1              |
|   total_timesteps   | 2048           |
-----

```

Eval num_timesteps=4000, episode_reward=102.00 +/- 48.63

Episode length: 102.00 +/- 48.63

```

-----
| eval/                |                |

```


	mean_ep_length		102	
	mean_reward		102	
	time/			
	total_timesteps		4000	
	train/			
	approx_kl		0.009253694	
	clip_fraction		0.11	
	clip_range		0.2	
	entropy_loss		-0.686	
	explained_variance		-0.00124	
	learning_rate		0.0003	
	loss		6.89	
	n_updates		10	
	policy_gradient_loss		-0.0156	
	value_loss		51.9	

New best mean reward!

	rollout/			
	ep_len_mean		26.8	
	ep_rew_mean		26.8	
	time/			
	fps		801	
	iterations		2	
	time_elapsed		5	
	total_timesteps		4096	

Eval num_timesteps=6000, episode_reward=224.20 +/- 57.20
Episode length: 224.20 +/- 57.20

	eval/			
	mean_ep_length		224	
	mean_reward		224	
	time/			
	total_timesteps		6000	
	train/			
	approx_kl		0.010190729	
	clip_fraction		0.0642	
	clip_range		0.2	
	entropy_loss		-0.664	
	explained_variance		0.11	
	learning_rate		0.0003	
	loss		14.9	
	n_updates		20	
	policy_gradient_loss		-0.0159	
	value_loss		33.3	

New best mean reward!

```

-----
| rollout/          |          |
|   ep_len_mean    | 36       |
|   ep_rew_mean    | 36       |
| time/            |          |
|   fps            | 677      |
|   iterations     | 3        |
|   time_elapsed   | 9        |
|   total_timesteps | 6144     |
-----

```

Eval num_timesteps=8000, episode_reward=358.60 +/- 157.75
 Episode length: 358.60 +/- 157.75

```

-----
| eval/              |          |
|   mean_ep_length  | 359      |
|   mean_reward     | 359      |
| time/              |          |
|   total_timesteps | 8000     |
| train/             |          |
|   approx_kl       | 0.007514276 |
|   clip_fraction   | 0.065    |
|   clip_range      | 0.2      |
|   entropy_loss    | -0.635   |
|   explained_variance | 0.231    |
|   learning_rate    | 0.0003   |
|   loss            | 18.1     |
|   n_updates       | 30       |
|   policy_gradient_loss | -0.0167  |
|   value_loss      | 54.7     |
-----

```

New best mean reward!

```

-----
| rollout/          |          |
|   ep_len_mean    | 44.5     |
|   ep_rew_mean    | 44.5     |
| time/            |          |
|   fps            | 605      |
|   iterations     | 4        |
|   time_elapsed   | 13       |
|   total_timesteps | 8192     |
-----

```

Eval num_timesteps=10000, episode_reward=286.60 +/- 174.30
 Episode length: 286.60 +/- 174.30

```

-----
| eval/              |          |
|   mean_ep_length  | 287      |
|   mean_reward     | 287      |
| time/              |          |

```

	total_timesteps		10000	
	train/			
	approx_kl		0.007736101	
	clip_fraction		0.0656	
	clip_range		0.2	
	entropy_loss		-0.602	
	explained_variance		0.329	
	learning_rate		0.0003	
	loss		28.7	
	n_updates		40	
	policy_gradient_loss		-0.0172	
	value_loss		63.8	

	rollout/			
	ep_len_mean		61.6	
	ep_rew_mean		61.6	
	time/			
	fps		583	
	iterations		5	
	time_elapsed		17	
	total_timesteps		10240	

Eval num_timesteps=12000, episode_reward=414.60 +/- 97.17
Episode length: 414.60 +/- 97.17

	eval/			
	mean_ep_length		415	
	mean_reward		415	
	time/			
	total_timesteps		12000	
	train/			
	approx_kl		0.005971997	
	clip_fraction		0.048	
	clip_range		0.2	
	entropy_loss		-0.609	
	explained_variance		0.391	
	learning_rate		0.0003	
	loss		28.7	
	n_updates		50	
	policy_gradient_loss		-0.0117	
	value_loss		57.4	

New best mean reward!

	rollout/			
	ep_len_mean		75.3	
	ep_rew_mean		75.3	

time/		
fps	555	
iterations	6	
time_elapsed	22	
total_timesteps	12288	

Eval num_timesteps=14000, episode_reward=500.00 +/- 0.00
 Episode length: 500.00 +/- 0.00

eval/		
mean_ep_length	500	
mean_reward	500	
time/		
total_timesteps	14000	
train/		
approx_kl	0.0075852736	
clip_fraction	0.0765	
clip_range	0.2	
entropy_loss	-0.595	
explained_variance	0.536	
learning_rate	0.0003	
loss	14	
n_updates	60	
policy_gradient_loss	-0.0133	
value_loss	58.1	

New best mean reward!

rollout/		
ep_len_mean	91.3	
ep_rew_mean	91.3	
time/		
fps	535	
iterations	7	
time_elapsed	26	
total_timesteps	14336	

Eval num_timesteps=16000, episode_reward=482.40 +/- 35.20
 Episode length: 482.40 +/- 35.20

eval/		
mean_ep_length	482	
mean_reward	482	
time/		
total_timesteps	16000	
train/		
approx_kl	0.0051994557	
clip_fraction	0.0533	

clip_range	0.2	
entropy_loss	-0.584	
explained_variance	0.716	
learning_rate	0.0003	
loss	30.6	
n_updates	70	
policy_gradient_loss	-0.00791	
value_loss	49.3	

rollout/		
ep_len_mean	109	
ep_rew_mean	109	
time/		
fps	522	
iterations	8	
time_elapsed	31	
total_timesteps	16384	

Eval num_timesteps=18000, episode_reward=451.80 +/- 62.30
Episode length: 451.80 +/- 62.30

eval/		
mean_ep_length	452	
mean_reward	452	
time/		
total_timesteps	18000	
train/		
approx_kl	0.006806485	
clip_fraction	0.05	
clip_range	0.2	
entropy_loss	-0.572	
explained_variance	0.429	
learning_rate	0.0003	
loss	23.8	
n_updates	80	
policy_gradient_loss	-0.00888	
value_loss	68.2	

rollout/		
ep_len_mean	126	
ep_rew_mean	126	
time/		
fps	513	
iterations	9	
time_elapsed	35	
total_timesteps	18432	

```
-----
Eval num_timesteps=20000, episode_reward=447.40 +/- 34.63
Episode length: 447.40 +/- 34.63
-----
```

eval/		
mean_ep_length	447	
mean_reward	447	
time/		
total_timesteps	20000	
train/		
approx_kl	0.008621543	
clip_fraction	0.0867	
clip_range	0.2	
entropy_loss	-0.572	
explained_variance	0.813	
learning_rate	0.0003	
loss	3.71	
n_updates	90	
policy_gradient_loss	-0.0137	
value_loss	32	

```
-----
```

rollout/		
ep_len_mean	144	
ep_rew_mean	144	
time/		
fps	507	
iterations	10	
time_elapsed	40	
total_timesteps	20480	

```
-----
```

Modelos guardados en ./logs/

Archivos: ['evaluations.npz', 'best_model.zip',
'ppo_cartpole_5000_steps.zip', 'ppo_cartpole_10000_steps.zip',
'ppo_cartpole_15000_steps.zip', 'ppo_cartpole_20000_steps.zip']

```
[10]: # TENSORBOARD PARA VISUALIZACIÓN
# =====

if SB3_AVAILABLE:
    # SB3 integra automáticamente con TensorBoard

    model = PPO(
        "MlpPolicy",
        "CartPole-v1",
        verbose=1,
```

```

    tensorboard_log="./tensorboard_logs/" # Directorio de logs
)

# Entrenar (los logs se guardan automáticamente)
model.learn(total_timesteps=10_000)

print("\n Para visualizar en TensorBoard:")
print("    tensorboard --logdir=./tensorboard_logs/")
print("    Luego abre http://localhost:6006 en tu navegador")

```

Using cpu device

Creating environment from the given name 'CartPole-v1'

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

Logging to ./tensorboard_logs/PP0_2

```

-----
| rollout/          |          |
|   ep_len_mean    | 27.2     |
|   ep_rew_mean    | 27.2     |
| time/            |          |
|   fps            | 1625     |
|   iterations     | 1        |
|   time_elapsed   | 1        |
|   total_timesteps | 2048     |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 27.8     |
|   ep_rew_mean    | 27.8     |
| time/            |          |
|   fps            | 869      |
|   iterations     | 2        |
|   time_elapsed   | 4        |
|   total_timesteps | 4096     |
| train/           |          |
|   approx_kl      | 0.007210049 |
|   clip_fraction  | 0.0658    |
|   clip_range     | 0.2       |
|   entropy_loss   | -0.687    |
|   explained_variance | 0.0112    |
|   learning_rate  | 0.0003    |
|   loss           | 8.29      |
|   n_updates      | 10        |
|   policy_gradient_loss | -0.00977 |
|   value_loss     | 67.3      |
-----

```

rollout/		
ep_len_mean	31.6	
ep_rew_mean	31.6	
time/		
fps	761	
iterations	3	
time_elapsed	8	
total_timesteps	6144	
train/		
approx_kl	0.010477221	
clip_fraction	0.0691	
clip_range	0.2	
entropy_loss	-0.667	
explained_variance	0.0607	
learning_rate	0.0003	
loss	12.9	
n_updates	20	
policy_gradient_loss	-0.0179	
value_loss	34.4	

rollout/		
ep_len_mean	46	
ep_rew_mean	46	
time/		
fps	706	
iterations	4	
time_elapsed	11	
total_timesteps	8192	
train/		
approx_kl	0.013019058	
clip_fraction	0.152	
clip_range	0.2	
entropy_loss	-0.631	
explained_variance	0.373	
learning_rate	0.0003	
loss	13.8	
n_updates	30	
policy_gradient_loss	-0.0285	
value_loss	40	

rollout/		
ep_len_mean	60	
ep_rew_mean	60	
time/		
fps	680	
iterations	5	

	time_elapsed	15	
	total_timesteps	10240	
	train/		
	approx_kl	0.009987105	
	clip_fraction	0.0636	
	clip_range	0.2	
	entropy_loss	-0.611	
	explained_variance	0.391	
	learning_rate	0.0003	
	loss	23.9	
	n_updates	40	
	policy_gradient_loss	-0.014	
	value_loss	59.2	

Para visualizar en TensorBoard:

tensorboard --logdir=./tensorboard_logs/

Luego abre <http://localhost:6006> en tu navegador

```
[11]: # GUARDAR Y CARGAR MODELOS
# =====

if SB3_AVAILABLE:
    # Entrenar un modelo
    model = PPO("MlpPolicy", "CartPole-v1", verbose=0)
    model.learn(total_timesteps=10_000)

    # GUARDAR
    # -----
    model.save("ppo_cartpole") # Guarda como ppo_cartpole.zip
    print(" Modelo guardado: ppo_cartpole.zip")

    # Eliminar el modelo de memoria
    del model

    # CARGAR
    # -----
    model = PPO.load("ppo_cartpole") # Carga desde ppo_cartpole.zip
    print(" Modelo cargado")

    # Verificar que funciona
    env = gym.make("CartPole-v1")
    mean_r, _ = evaluate_policy(model, env, n_eval_episodes=5)
    print(f" Recompensa tras cargar: {mean_r:.1f}")

    # CONTINUAR ENTRENAMIENTO
    # -----
```

```

# Necesitas pasar el entorno al cargar
model = PPO.load("ppo_cartpole", env=gym.make("CartPole-v1"))
model.learn(total_timesteps=5_000) # Continúa entrenando
print(" Entrenamiento continuado")

```

Modelo guardado: ppo_cartpole.zip
 Modelo cargado
 Recompensa tras cargar: 368.4
 Entrenamiento continuado

```

[12]: # EVALUACIÓN COMPLETA
# =====

if SB3_AVAILABLE:
    from stable_baselines3.common.evaluation import evaluate_policy

    # Entrenar modelo
    model = PPO("MlpPolicy", "CartPole-v1", verbose=0)
    model.learn(total_timesteps=20_000)

    # Crear entorno de evaluación
    eval_env = gym.make("CartPole-v1")

    # Evaluar con estadísticas
    mean_reward, std_reward = evaluate_policy(
        model,
        eval_env,
        n_eval_episodes=20,          # Número de episodios
        deterministic=True,         # Acciones deterministas (sin exploración)
        return_episode_rewards=False
    )

    print(f" Evaluación (20 episodios):")
    print(f"   Recompensa media: {mean_reward:.2f}")
    print(f"   Desviación estándar: {std_reward:.2f}")
    print(f"   Rango esperado: [{mean_reward-2*std_reward:.1f}, ↵
↵{mean_reward+2*std_reward:.1f}]" )

```

Evaluación (20 episodios):
 Recompensa media: 500.00
 Desviación estándar: 0.00
 Rango esperado: [500.0, 500.0]

```

[13]: # EVALUACIÓN DETALLADA CON HISTORIAL
# =====

if SB3_AVAILABLE:
    # Obtener recompensas individuales

```

```

episode_rewards, episode_lengths = evaluate_policy(
    model,
    eval_env,
    n_eval_episodes=20,
    return_episode_rewards=True
)

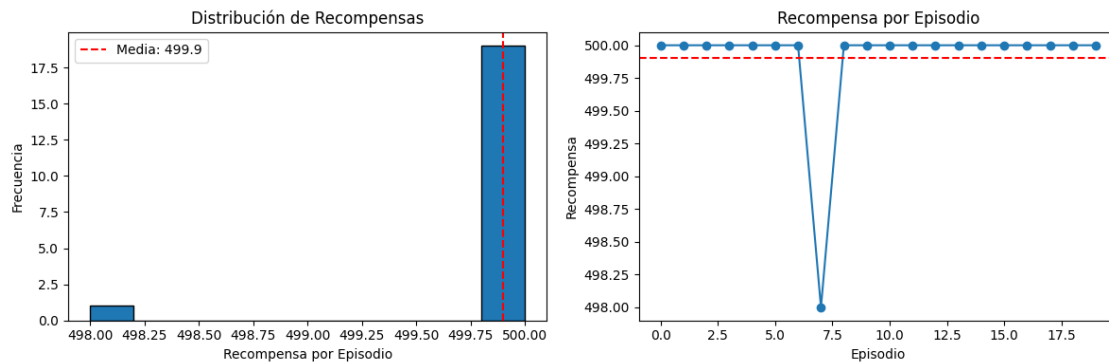
# Visualizar distribución
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(episode_rewards, bins=10, edgecolor='black')
axes[0].axvline(np.mean(episode_rewards), color='red', linestyle='--',
label=f'Media: {np.mean(episode_rewards):.1f}')
axes[0].set_xlabel("Recompensa por Episodio")
axes[0].set_ylabel("Frecuencia")
axes[0].set_title("Distribución de Recompensas")
axes[0].legend()

axes[1].plot(episode_rewards, 'o-')
axes[1].axhline(np.mean(episode_rewards), color='red', linestyle='--')
axes[1].set_xlabel("Episodio")
axes[1].set_ylabel("Recompensa")
axes[1].set_title("Recompensa por Episodio")

plt.tight_layout()
plt.show()

```



```

[14]: # EJEMPLO: PPO CON HIPERPARÁMETROS CUSTOM
# =====

if SB3_AVAILABLE:
    model = PPO(
        "MlpPolicy",

```

```

    "CartPole-v1",

    # Hiperparámetros de aprendizaje
    learning_rate=1e-3,      # Más alto = aprende más rápido pero menos
    ↪ estable
    gamma=0.99,              # Factor de descuento

    # Hiperparámetros de muestreo
    n_steps=1024,            # Pasos antes de cada actualización
    batch_size=64,           # Tamaño del minibatch
    n_epochs=10,             # Épocas de optimización

    # Hiperparámetros de PPO específicos
    clip_range=0.2,          # Rango de clipping
    ent_coef=0.01,           # Bonus por entropía (promueve exploración)

    # Otros
    verbose=1
)

model.learn(total_timesteps=20_000)

mean_r, _ = evaluate_policy(model, gym.make("CartPole-v1"),
    ↪ n_eval_episodes=10)
print(f"\n Resultado: {mean_r:.1f}")

```

Using cpu device

Creating environment from the given name 'CartPole-v1'

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

```

-----
| rollout/          |          |
|   ep_len_mean    | 20.2     |
|   ep_rew_mean    | 20.2     |
| time/            |          |
|   fps            | 1652     |
|   iterations     | 1         |
|   time_elapsed   | 0         |
|   total_timesteps | 1024      |
-----

```

```

-----
| rollout/          |          |
|   ep_len_mean    | 23.6     |
|   ep_rew_mean    | 23.6     |
| time/            |          |
|   fps            | 846      |
|   iterations     | 2         |

```

	time_elapsed		2	
	total_timesteps		2048	
	train/			
	approx_kl		0.012711421	
	clip_fraction		0.161	
	clip_range		0.2	
	entropy_loss		-0.684	
	explained_variance		0.00308	
	learning_rate		0.001	
	loss		4.98	
	n_updates		10	
	policy_gradient_loss		-0.0201	
	value_loss		35.9	

	rollout/			
	ep_len_mean		29.2	
	ep_rew_mean		29.2	
	time/			
	fps		704	
	iterations		3	
	time_elapsed		4	
	total_timesteps		3072	
	train/			
	approx_kl		0.011307549	
	clip_fraction		0.132	
	clip_range		0.2	
	entropy_loss		-0.663	
	explained_variance		0.283	
	learning_rate		0.001	
	loss		13.8	
	n_updates		20	
	policy_gradient_loss		-0.0241	
	value_loss		30.1	

	rollout/			
	ep_len_mean		36.8	
	ep_rew_mean		36.8	
	time/			
	fps		677	
	iterations		4	
	time_elapsed		6	
	total_timesteps		4096	
	train/			
	approx_kl		0.008096636	
	clip_fraction		0.0963	
	clip_range		0.2	

	entropy_loss		-0.632	
	explained_variance		0.383	
	learning_rate		0.001	
	loss		18.6	
	n_updates		30	
	policy_gradient_loss		-0.0236	
	value_loss		44.3	

	rollout/			
	ep_len_mean		45.4	
	ep_rew_mean		45.4	
	time/			
	fps		658	
	iterations		5	
	time_elapsed		7	
	total_timesteps		5120	
	train/			
	approx_kl		0.011208512	
	clip_fraction		0.0956	
	clip_range		0.2	
	entropy_loss		-0.607	
	explained_variance		0.518	
	learning_rate		0.001	
	loss		7.37	
	n_updates		40	
	policy_gradient_loss		-0.0156	
	value_loss		43.7	

	rollout/			
	ep_len_mean		54	
	ep_rew_mean		54	
	time/			
	fps		640	
	iterations		6	
	time_elapsed		9	
	total_timesteps		6144	
	train/			
	approx_kl		0.0029176855	
	clip_fraction		0.0793	
	clip_range		0.2	
	entropy_loss		-0.605	
	explained_variance		0.547	
	learning_rate		0.001	
	loss		11.8	
	n_updates		50	
	policy_gradient_loss		-0.0117	

	value_loss	40.2	

	rollout/		
	ep_len_mean	64.2	
	ep_rew_mean	64.2	
	time/		
	fps	632	
	iterations	7	
	time_elapsed	11	
	total_timesteps	7168	
	train/		
	approx_kl	0.013249515	
	clip_fraction	0.163	
	clip_range	0.2	
	entropy_loss	-0.582	
	explained_variance	0.762	
	learning_rate	0.001	
	loss	2.3	
	n_updates	60	
	policy_gradient_loss	-0.0147	
	value_loss	22.7	

	rollout/		
	ep_len_mean	72.3	
	ep_rew_mean	72.3	
	time/		
	fps	620	
	iterations	8	
	time_elapsed	13	
	total_timesteps	8192	
	train/		
	approx_kl	0.010470515	
	clip_fraction	0.114	
	clip_range	0.2	
	entropy_loss	-0.573	
	explained_variance	0.891	
	learning_rate	0.001	
	loss	2.92	
	n_updates	70	
	policy_gradient_loss	-0.0171	
	value_loss	15.7	

	rollout/		
	ep_len_mean	82	
	ep_rew_mean	82	

time/		
fps	615	
iterations	9	
time_elapsed	14	
total_timesteps	9216	
train/		
approx_kl	0.0052982047	
clip_fraction	0.0686	
clip_range	0.2	
entropy_loss	-0.546	
explained_variance	0.55	
learning_rate	0.001	
loss	17.5	
n_updates	80	
policy_gradient_loss	-0.0103	
value_loss	61.7	

rollout/		
ep_len_mean	91.8	
ep_rew_mean	91.8	
time/		
fps	610	
iterations	10	
time_elapsed	16	
total_timesteps	10240	
train/		
approx_kl	0.012478186	
clip_fraction	0.152	
clip_range	0.2	
entropy_loss	-0.549	
explained_variance	0.948	
learning_rate	0.001	
loss	0.831	
n_updates	90	
policy_gradient_loss	-0.0174	
value_loss	7.14	

rollout/		
ep_len_mean	101	
ep_rew_mean	101	
time/		
fps	608	
iterations	11	
time_elapsed	18	
total_timesteps	11264	
train/		

approx_kl	0.015154382	
clip_fraction	0.127	
clip_range	0.2	
entropy_loss	-0.531	
explained_variance	0.282	
learning_rate	0.001	
loss	0.501	
n_updates	100	
policy_gradient_loss	-0.00812	
value_loss	4.24	

rollout/		
ep_len_mean	111	
ep_rew_mean	111	
time/		
fps	605	
iterations	12	
time_elapsed	20	
total_timesteps	12288	
train/		
approx_kl	0.015350283	
clip_fraction	0.27	
clip_range	0.2	
entropy_loss	-0.542	
explained_variance	0.469	
learning_rate	0.001	
loss	0.55	
n_updates	110	
policy_gradient_loss	-0.0189	
value_loss	3.23	

rollout/		
ep_len_mean	121	
ep_rew_mean	121	
time/		
fps	604	
iterations	13	
time_elapsed	22	
total_timesteps	13312	
train/		
approx_kl	0.0064184275	
clip_fraction	0.0939	
clip_range	0.2	
entropy_loss	-0.515	
explained_variance	-0.115	
learning_rate	0.001	

loss	0.0789
n_updates	120
policy_gradient_loss	-0.00159
value_loss	1.63

rollout/	
ep_len_mean	130
ep_rew_mean	130
time/	
fps	604
iterations	14
time_elapsed	23
total_timesteps	14336
train/	
approx_kl	0.004459546
clip_fraction	0.0533
clip_range	0.2
entropy_loss	-0.498
explained_variance	-0.0651
learning_rate	0.001
loss	0.139
n_updates	130
policy_gradient_loss	-0.00305
value_loss	1.08

rollout/	
ep_len_mean	140
ep_rew_mean	140
time/	
fps	603
iterations	15
time_elapsed	25
total_timesteps	15360
train/	
approx_kl	0.0053522633
clip_fraction	0.0469
clip_range	0.2
entropy_loss	-0.509
explained_variance	-0.0322
learning_rate	0.001
loss	0.138
n_updates	140
policy_gradient_loss	0.000966
value_loss	0.697

rollout/		
ep_len_mean	149	
ep_rew_mean	149	
time/		
fps	601	
iterations	16	
time_elapsed	27	
total_timesteps	16384	
train/		
approx_kl	0.005950732	
clip_fraction	0.0928	
clip_range	0.2	
entropy_loss	-0.522	
explained_variance	0.315	
learning_rate	0.001	
loss	0.00348	
n_updates	150	
policy_gradient_loss	-0.00794	
value_loss	0.484	

rollout/		
ep_len_mean	159	
ep_rew_mean	159	
time/		
fps	597	
iterations	17	
time_elapsed	29	
total_timesteps	17408	
train/		
approx_kl	0.0039026178	
clip_fraction	0.0309	
clip_range	0.2	
entropy_loss	-0.498	
explained_variance	-0.0724	
learning_rate	0.001	
loss	0.0349	
n_updates	160	
policy_gradient_loss	-0.00029	
value_loss	0.281	

rollout/		
ep_len_mean	168	
ep_rew_mean	168	
time/		
fps	598	
iterations	18	

	time_elapsed		30	
	total_timesteps		18432	
	train/			
	approx_kl		0.004516681	
	clip_fraction		0.0738	
	clip_range		0.2	
	entropy_loss		-0.495	
	explained_variance		-0.0301	
	learning_rate		0.001	
	loss		-0.0144	
	n_updates		170	
	policy_gradient_loss		-0.00398	
	value_loss		0.196	

	rollout/			
	ep_len_mean		178	
	ep_rew_mean		178	
	time/			
	fps		597	
	iterations		19	
	time_elapsed		32	
	total_timesteps		19456	
	train/			
	approx_kl		0.009455321	
	clip_fraction		0.0783	
	clip_range		0.2	
	entropy_loss		-0.511	
	explained_variance		-0.027	
	learning_rate		0.001	
	loss		0.0242	
	n_updates		180	
	policy_gradient_loss		-0.00386	
	value_loss		0.131	

	rollout/			
	ep_len_mean		187	
	ep_rew_mean		187	
	time/			
	fps		597	
	iterations		20	
	time_elapsed		34	
	total_timesteps		20480	
	train/			
	approx_kl		0.0051630763	
	clip_fraction		0.0682	
	clip_range		0.2	

	entropy_loss		-0.523	
	explained_variance		0.0149	
	learning_rate		0.001	
	loss		-0.00793	
	n_updates		190	
	policy_gradient_loss		-0.00667	
	value_loss		0.0895	

Resultado: 500.0

```
[15]: # EJEMPLO: LUNARLANDER
# =====
# Este es uno de los entornos más populares para aprender RL

if SB3_AVAILABLE:
    try:
        # Crear entorno (requiere gymnasium[box2d])
        env = gym.make("LunarLander-v3")

        print(" LunarLander-v3")
        print(f"  Observaciones: {env.observation_space}")
        print(f"  Acciones: {env.action_space}")
        print(f"  Acciones disponibles:")
        print(f"    0: No hacer nada")
        print(f"    1: Motor izquierdo")
        print(f"    2: Motor principal")
        print(f"    3: Motor derecho")

        # Entrenar con PPO
        print("\n Entrenando PPO (esto toma ~1-2 minutos)...")
        model = PPO("MlpPolicy", env, verbose=1)
        model.learn(total_timesteps=50_000)

        # Evaluar
        mean_r, std_r = evaluate_policy(model, env, n_eval_episodes=10)
        print(f"\n Resultado: {mean_r:.1f} ± {std_r:.1f}")
        print(f"    (>200 = aterrizaje exitoso)")

        env.close()

    except Exception as e:
        print(f" Error: {e}")
        print("  Instala Box2D: pip install gymnasium[box2d]")
```

```
LunarLander-v3
Observaciones: Box([ -2.5          -2.5          -10.          -10.
-6.2831855 -10.]
```

```
-0.          -0.          ], [ 2.5          2.5          10.          10.
6.2831855 10.
```

```
1.          1.          ], (8,), float32)
```

```
Acciones: Discrete(4)
```

```
Acciones disponibles:
```

```
0: No hacer nada
```

```
1: Motor izquierdo
```

```
2: Motor principal
```

```
3: Motor derecho
```

Entrenando PPO (esto toma ~1-2 minutos)...

Using cpu device

Wrapping the env with a `Monitor` wrapper

Wrapping the env in a DummyVecEnv.

```
-----
| rollout/          |          |
|   ep_len_mean    | 92.9     |
|   ep_rew_mean    | -155     |
| time/           |          |
|   fps            | 1194     |
|   iterations     | 1        |
|   time_elapsed   | 1        |
|   total_timesteps | 2048     |
-----
```

```
-----
| rollout/          |          |
|   ep_len_mean    | 91.3     |
|   ep_rew_mean    | -151     |
| time/           |          |
|   fps            | 744      |
|   iterations     | 2        |
|   time_elapsed   | 5        |
|   total_timesteps | 4096     |
| train/          |          |
|   approx_kl      | 0.004589392 |
|   clip_fraction  | 0.00913   |
|   clip_range     | 0.2       |
|   entropy_loss   | -1.38     |
|   explained_variance | 0.0044    |
|   learning_rate  | 0.0003    |
|   loss           | 460       |
|   n_updates      | 10        |
|   policy_gradient_loss | -0.00506  |
|   value_loss     | 1.45e+03  |
-----
```

```
-----
| rollout/          |          |
|   ep_len_mean    | 93        |
|-----
```

	ep_rew_mean		-151	
	time/			
	fps		657	
	iterations		3	
	time_elapsed		9	
	total_timesteps		6144	
	train/			
	approx_kl		0.01089083	
	clip_fraction		0.0466	
	clip_range		0.2	
	entropy_loss		-1.38	
	explained_variance		-0.0146	
	learning_rate		0.0003	
	loss		310	
	n_updates		20	
	policy_gradient_loss		-0.00812	
	value_loss		1.24e+03	

	rollout/			
	ep_len_mean		95.1	
	ep_rew_mean		-148	
	time/			
	fps		616	
	iterations		4	
	time_elapsed		13	
	total_timesteps		8192	
	train/			
	approx_kl		0.010545075	
	clip_fraction		0.0371	
	clip_range		0.2	
	entropy_loss		-1.35	
	explained_variance		0.000923	
	learning_rate		0.0003	
	loss		322	
	n_updates		30	
	policy_gradient_loss		-0.0062	
	value_loss		715	

	rollout/			
	ep_len_mean		108	
	ep_rew_mean		-143	
	time/			
	fps		592	
	iterations		5	
	time_elapsed		17	
	total_timesteps		10240	

train/		
approx_kl	0.015775934	
clip_fraction	0.104	
clip_range	0.2	
entropy_loss	-1.34	
explained_variance	0.0412	
learning_rate	0.0003	
loss	201	
n_updates	40	
policy_gradient_loss	-0.0124	
value_loss	341	

rollout/		
ep_len_mean	109	
ep_rew_mean	-137	
time/		
fps	576	
iterations	6	
time_elapsed	21	
total_timesteps	12288	
train/		
approx_kl	0.008699062	
clip_fraction	0.0658	
clip_range	0.2	
entropy_loss	-1.29	
explained_variance	-0.0474	
learning_rate	0.0003	
loss	232	
n_updates	50	
policy_gradient_loss	-0.00925	
value_loss	707	

rollout/		
ep_len_mean	124	
ep_rew_mean	-129	
time/		
fps	570	
iterations	7	
time_elapsed	25	
total_timesteps	14336	
train/		
approx_kl	0.009405017	
clip_fraction	0.0456	
clip_range	0.2	
entropy_loss	-1.28	
explained_variance	0.396	

	learning_rate		0.0003	
	loss		38.5	
	n_updates		60	
	policy_gradient_loss		-0.00531	
	value_loss		170	

	rollout/			
	ep_len_mean		131	
	ep_rew_mean		-120	
	time/			
	fps		565	
	iterations		8	
	time_elapsed		28	
	total_timesteps		16384	
	train/			
	approx_kl		0.0071712816	
	clip_fraction		0.0829	
	clip_range		0.2	
	entropy_loss		-1.23	
	explained_variance		-0.365	
	learning_rate		0.0003	
	loss		111	
	n_updates		70	
	policy_gradient_loss		-0.00302	
	value_loss		359	

	rollout/			
	ep_len_mean		137	
	ep_rew_mean		-115	
	time/			
	fps		563	
	iterations		9	
	time_elapsed		32	
	total_timesteps		18432	
	train/			
	approx_kl		0.00988942	
	clip_fraction		0.145	
	clip_range		0.2	
	entropy_loss		-1.16	
	explained_variance		-0.00595	
	learning_rate		0.0003	
	loss		125	
	n_updates		80	
	policy_gradient_loss		-0.0117	
	value_loss		227	

rollout/		
ep_len_mean	151	
ep_rew_mean	-107	
time/		
fps	563	
iterations	10	
time_elapsed	36	
total_timesteps	20480	
train/		
approx_kl	0.0041403566	
clip_fraction	0.017	
clip_range	0.2	
entropy_loss	-1.2	
explained_variance	-0.0477	
learning_rate	0.0003	
loss	143	
n_updates	90	
policy_gradient_loss	-0.00533	
value_loss	351	

rollout/		
ep_len_mean	160	
ep_rew_mean	-104	
time/		
fps	562	
iterations	11	
time_elapsed	40	
total_timesteps	22528	
train/		
approx_kl	0.009630702	
clip_fraction	0.0443	
clip_range	0.2	
entropy_loss	-1.18	
explained_variance	0.133	
learning_rate	0.0003	
loss	164	
n_updates	100	
policy_gradient_loss	-0.00938	
value_loss	291	

rollout/		
ep_len_mean	173	
ep_rew_mean	-98.5	
time/		
fps	560	

	iterations		12	
	time_elapsed		43	
	total_timesteps		24576	
	train/			
	approx_kl		0.008989531	
	clip_fraction		0.0324	
	clip_range		0.2	
	entropy_loss		-1.16	
	explained_variance		-0.152	
	learning_rate		0.0003	
	loss		144	
	n_updates		110	
	policy_gradient_loss		-0.00694	
	value_loss		344	

	rollout/			
	ep_len_mean		174	
	ep_rew_mean		-104	
	time/			
	fps		558	
	iterations		13	
	time_elapsed		47	
	total_timesteps		26624	
	train/			
	approx_kl		0.00557749	
	clip_fraction		0.0427	
	clip_range		0.2	
	entropy_loss		-1.15	
	explained_variance		0.0253	
	learning_rate		0.0003	
	loss		81.4	
	n_updates		120	
	policy_gradient_loss		-0.00667	
	value_loss		288	

	rollout/			
	ep_len_mean		188	
	ep_rew_mean		-98.9	
	time/			
	fps		551	
	iterations		14	
	time_elapsed		51	
	total_timesteps		28672	
	train/			
	approx_kl		0.0055615855	
	clip_fraction		0.0258	

clip_range	0.2	
entropy_loss	-1.1	
explained_variance	-0.0286	
learning_rate	0.0003	
loss	99.3	
n_updates	130	
policy_gradient_loss	-0.00401	
value_loss	447	

rollout/		
ep_len_mean	201	
ep_rew_mean	-96.9	
time/		
fps	549	
iterations	15	
time_elapsed	55	
total_timesteps	30720	
train/		
approx_kl	0.007451481	
clip_fraction	0.0506	
clip_range	0.2	
entropy_loss	-1.12	
explained_variance	0.0998	
learning_rate	0.0003	
loss	90.2	
n_updates	140	
policy_gradient_loss	-0.004	
value_loss	229	

rollout/		
ep_len_mean	209	
ep_rew_mean	-94.2	
time/		
fps	548	
iterations	16	
time_elapsed	59	
total_timesteps	32768	
train/		
approx_kl	0.0072099213	
clip_fraction	0.0402	
clip_range	0.2	
entropy_loss	-1.17	
explained_variance	-0.0127	
learning_rate	0.0003	
loss	282	
n_updates	150	

	policy_gradient_loss		-0.00636	
	value_loss		360	

	rollout/			
	ep_len_mean		215	
	ep_rew_mean		-91.6	
	time/			
	fps		546	
	iterations		17	
	time_elapsed		63	
	total_timesteps		34816	
	train/			
	approx_kl		0.0060287016	
	clip_fraction		0.0192	
	clip_range		0.2	
	entropy_loss		-1.13	
	explained_variance		-0.0208	
	learning_rate		0.0003	
	loss		167	
	n_updates		160	
	policy_gradient_loss		-0.00425	
	value_loss		314	

	rollout/			
	ep_len_mean		223	
	ep_rew_mean		-91.6	
	time/			
	fps		545	
	iterations		18	
	time_elapsed		67	
	total_timesteps		36864	
	train/			
	approx_kl		0.009651601	
	clip_fraction		0.112	
	clip_range		0.2	
	entropy_loss		-1.08	
	explained_variance		0.0738	
	learning_rate		0.0003	
	loss		65.8	
	n_updates		170	
	policy_gradient_loss		-0.0122	
	value_loss		152	

	rollout/			
	ep_len_mean		233	

	ep_rew_mean		-90.1	
	time/			
	fps		545	
	iterations		19	
	time_elapsed		71	
	total_timesteps		38912	
	train/			
	approx_kl		0.008872478	
	clip_fraction		0.0533	
	clip_range		0.2	
	entropy_loss		-1.03	
	explained_variance		0.0461	
	learning_rate		0.0003	
	loss		213	
	n_updates		180	
	policy_gradient_loss		-0.00674	
	value_loss		370	

	rollout/			
	ep_len_mean		246	
	ep_rew_mean		-88.9	
	time/			
	fps		546	
	iterations		20	
	time_elapsed		74	
	total_timesteps		40960	
	train/			
	approx_kl		0.004208651	
	clip_fraction		0.0176	
	clip_range		0.2	
	entropy_loss		-1.03	
	explained_variance		0.0662	
	learning_rate		0.0003	
	loss		111	
	n_updates		190	
	policy_gradient_loss		-0.00419	
	value_loss		204	

	rollout/			
	ep_len_mean		261	
	ep_rew_mean		-84.6	
	time/			
	fps		543	
	iterations		21	
	time_elapsed		79	
	total_timesteps		43008	

train/		
approx_kl	0.0070110466	
clip_fraction	0.042	
clip_range	0.2	
entropy_loss	-0.955	
explained_variance	0.386	
learning_rate	0.0003	
loss	40.7	
n_updates	200	
policy_gradient_loss	-0.00583	
value_loss	128	

rollout/		
ep_len_mean	275	
ep_rew_mean	-85.6	
time/		
fps	543	
iterations	22	
time_elapsed	82	
total_timesteps	45056	
train/		
approx_kl	0.013395205	
clip_fraction	0.1	
clip_range	0.2	
entropy_loss	-0.939	
explained_variance	0.45	
learning_rate	0.0003	
loss	14.4	
n_updates	210	
policy_gradient_loss	-0.0122	
value_loss	99.6	

rollout/		
ep_len_mean	291	
ep_rew_mean	-84.7	
time/		
fps	542	
iterations	23	
time_elapsed	86	
total_timesteps	47104	
train/		
approx_kl	0.008858584	
clip_fraction	0.0773	
clip_range	0.2	
entropy_loss	-0.984	
explained_variance	0.303	

	learning_rate		0.0003	
	loss		59.6	
	n_updates		220	
	policy_gradient_loss		-0.00613	
	value_loss		178	

	rollout/			
	ep_len_mean		299	
	ep_rew_mean		-82.7	
	time/			
	fps		542	
	iterations		24	
	time_elapsed		90	
	total_timesteps		49152	
	train/			
	approx_kl		0.009269721	
	clip_fraction		0.0688	
	clip_range		0.2	
	entropy_loss		-1.01	
	explained_variance		0.381	
	learning_rate		0.0003	
	loss		59	
	n_updates		230	
	policy_gradient_loss		-0.0074	
	value_loss		124	

	rollout/			
	ep_len_mean		310	
	ep_rew_mean		-80.8	
	time/			
	fps		539	
	iterations		25	
	time_elapsed		94	
	total_timesteps		51200	
	train/			
	approx_kl		0.007742923	
	clip_fraction		0.0779	
	clip_range		0.2	
	entropy_loss		-1.01	
	explained_variance		0.538	
	learning_rate		0.0003	
	loss		44.1	
	n_updates		240	
	policy_gradient_loss		-0.00811	
	value_loss		90.6	

Resultado: -403.4 ± 102.2
(>200 = aterrizaje exitoso)

```
[16]: # EJEMPLO: ACCIONES CONTINUAS CON SAC
# =====

if SB3_AVAILABLE:
    try:
        # Pendulum tiene acciones continuas
        env = gym.make("Pendulum-v1")

        print(" Pendulum-v1")
        print(f"  Observaciones: {env.observation_space}")
        print(f"  Acciones: {env.action_space}")
        print(f"  Acción: torque continuo en [-2, 2]")

        # SAC es ideal para acciones continuas
        print("\n Entrenando SAC...")
        model = SAC("MlpPolicy", env, verbose=1)
        model.learn(total_timesteps=10_000)

        # Evaluar
        mean_r, std_r = evaluate_policy(model, env, n_eval_episodes=10)
        print(f"\n Resultado: {mean_r:.1f} ± {std_r:.1f}")
        print(f"  (Máximo teórico: 0, más negativo = peor)")

        env.close()

    except Exception as e:
        print(f" Error: {e}")
```

Pendulum-v1
Observaciones: Box([-1. -1. -8.], [1. 1. 8.], (3,), float32)
Acciones: Box(-2.0, 2.0, (1,), float32)
Acción: torque continuo en [-2, 2]

Entrenando SAC...
Using cpu device
Wrapping the env with a `Monitor` wrapper
Wrapping the env in a DummyVecEnv.

```
-----
| rollout/          |          |
|   ep_len_mean    | 200      |
|   ep_rew_mean    | -1.15e+03 |
| time/            |          |
|   episodes       | 4        |
|   fps            | 29       |
```

	time_elapsed	27	
	total_timesteps	800	
	train/		
	actor_loss	18.8	
	critic_loss	0.254	
	ent_coef	0.813	
	ent_coef_loss	-0.32	
	learning_rate	0.0003	
	n_updates	699	

	rollout/		
	ep_len_mean	200	
	ep_rew_mean	-1.37e+03	
	time/		
	episodes	8	
	fps	26	
	time_elapsed	60	
	total_timesteps	1600	
	train/		
	actor_loss	42	
	critic_loss	0.169	
	ent_coef	0.65	
	ent_coef_loss	-0.599	
	learning_rate	0.0003	
	n_updates	1499	

	rollout/		
	ep_len_mean	200	
	ep_rew_mean	-1.31e+03	
	time/		
	episodes	12	
	fps	28	
	time_elapsed	85	
	total_timesteps	2400	
	train/		
	actor_loss	62.6	
	critic_loss	0.219	
	ent_coef	0.53	
	ent_coef_loss	-0.65	
	learning_rate	0.0003	
	n_updates	2299	

	rollout/		
	ep_len_mean	200	
	ep_rew_mean	-1.11e+03	

time/		
episodes	16	
fps	31	
time_elapsed	102	
total_timesteps	3200	
train/		
actor_loss	68.9	
critic_loss	0.504	
ent_coef	0.453	
ent_coef_loss	-0.452	
learning_rate	0.0003	
n_updates	3099	

rollout/		
ep_len_mean	200	
ep_rew_mean	-913	
time/		
episodes	20	
fps	33	
time_elapsed	120	
total_timesteps	4000	
train/		
actor_loss	66.9	
critic_loss	0.719	
ent_coef	0.391	
ent_coef_loss	-0.488	
learning_rate	0.0003	
n_updates	3899	

rollout/		
ep_len_mean	200	
ep_rew_mean	-801	
time/		
episodes	24	
fps	34	
time_elapsed	137	
total_timesteps	4800	
train/		
actor_loss	67.6	
critic_loss	1.16	
ent_coef	0.33	
ent_coef_loss	-0.608	
learning_rate	0.0003	
n_updates	4699	

rollout/		
ep_len_mean	200	
ep_rew_mean	-717	
time/		
episodes	28	
fps	35	
time_elapsed	155	
total_timesteps	5600	
train/		
actor_loss	53.4	
critic_loss	0.946	
ent_coef	0.274	
ent_coef_loss	-0.575	
learning_rate	0.0003	
n_updates	5499	

rollout/		
ep_len_mean	200	
ep_rew_mean	-654	
time/		
episodes	32	
fps	36	
time_elapsed	173	
total_timesteps	6400	
train/		
actor_loss	63.9	
critic_loss	1.25	
ent_coef	0.229	
ent_coef_loss	-0.494	
learning_rate	0.0003	
n_updates	6299	

rollout/		
ep_len_mean	200	
ep_rew_mean	-597	
time/		
episodes	36	
fps	38	
time_elapsed	188	
total_timesteps	7200	
train/		
actor_loss	64.5	
critic_loss	1.56	
ent_coef	0.191	
ent_coef_loss	-0.542	
learning_rate	0.0003	

	n_updates		7099	

	rollout/			
	ep_len_mean		200	
	ep_rew_mean		-550	
	time/			
	episodes		40	
	fps		39	
	time_elapsed		204	
	total_timesteps		8000	
	train/			
	actor_loss		59.4	
	critic_loss		1.31	
	ent_coef		0.159	
	ent_coef_loss		-0.94	
	learning_rate		0.0003	
	n_updates		7899	

	rollout/			
	ep_len_mean		200	
	ep_rew_mean		-516	
	time/			
	episodes		44	
	fps		39	
	time_elapsed		221	
	total_timesteps		8800	
	train/			
	actor_loss		56.1	
	critic_loss		1.59	
	ent_coef		0.133	
	ent_coef_loss		-0.884	
	learning_rate		0.0003	
	n_updates		8699	

	rollout/			
	ep_len_mean		200	
	ep_rew_mean		-491	
	time/			
	episodes		48	
	fps		40	
	time_elapsed		236	
	total_timesteps		9600	
	train/			
	actor_loss		58.6	
	critic_loss		1.54	

	ent_coef		0.112	
	ent_coef_loss		-0.433	
	learning_rate		0.0003	
	n_updates		9499	

Resultado: -107.8 ± 98.0

(Máximo teórico: 0, más negativo = peor)